POLYGLOT WORKSHOP

LLM PRETRAINING

PART II

Tutors
Nicholas Kluge & Aniket Sen

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

AGENDA

1. How do We Know we are Doing Well?

- ✓ Why we need Evals
- ✓ Complications
- ✓ Picking an Evaluation Framework
- ✓ Benchmarks to Know
- ✓ Good Practices

2. A Closer Look at Evaluations

- ✓ Two Main Evaluation Approaches
- ✓ Creating Your Own Evals

3. Selecting "Good" Benchmarks

- ✓ What makes a benchmark “good”?

HOW DO WE KNOW WE ARE DOING WELL?

Evaluations

WHY WE NEED EVALS

How do we know whether a model is *good*?

- ✓ **We run evals.** Evals are everywhere: leaderboards ranking models, benchmarks claiming to measure *reasoning*, *knowledge*, *coding ability*, or *mathematical performance*, and papers announcing new state-of-the-art results.

- ✓ **They are our signal**. They define what we seek to optimize and improve. Fortunately, good evaluations usually **correlate with what we, as humans, care about in large language models**. For example, a model that performs well on GSM8K is generally **capable of basic mathematical reasoning** and can correctly answer elementary math problems.

WHY WE NEED EVALS

- ✓ For a base model trained from **scratch**, you want strong performance on a **broad set of tasks** that measure a wide range of capabilities.
- ✓ If you are **post-training** a base model for a specific use case, you likely care most about performance on that **particular task**.
- ✓ If you are continually pretraining, you want to ensure that domain adaptation does not ***break*** **existing capabilities**.
- ✓ More broadly, as you experiment with different architectures, data mixtures, and training recipes, you need to **verify that your changes have not degraded the model**.
- ✓ **Remember:** if you rely only on training loss to guide development (like a good old-fashioned machine learning engineer), you may **miss critical failures**.

COMPLICATIONS

Before 2022, most models were simply pretrained: **raw text in, raw text out**. In 2023, **instruction tuning and chat models** became standard. By 2025, we introduced **reasoning** models. Each step added new structure and **formatting constraints**. We moved from plain text to increasingly formatted inputs and outputs.

As a result, many models will perform poorly unless you carefully handle the following:

- ✔ **Respect the format the model expects.**
- ✔ **Include a system prompt at the very beginning of inference**, if the model requires one.
- ✔ **Remove thinking traces from reasoning model outputs**.
- ✔ **Be aware of tokenizer and special-token sensitivities**.

💡 **T**he **Gemma family** is extremely sensitive to the inclusion of start-of-sentence tokens at inference time

WHY WE NEED A UNIFIED EVALUATION FRAMEWORK

With all these complications—different formats, system prompts, reasoning traces, and token sensitivities—consistently evaluating models **becomes challenging**.

A unified framework lets us:

- ✔ Apply the **same evaluation method** to every model.
- ✔ Ensure **portability and reproducibility**.
- ✔ Avoid reinventing the wheel for every new experiment.

💡 Maintaining your own ad-hoc library of evaluation scripts is risky: it's **time-consuming, error-prone**, and **likely to become outdated** unless you commit to building a full evaluation framework.

PICKING AN EVALUATION FRAMEWORK

	LM-Evaluation-Harness	Lighteval
Organization	EleutherAI	Hugging Face
Maturity	Very mature (11.4k ☆, 399 contributors)	Newer (2.3k ☆, 128 contributors)
Adoption	Used in hundreds of papers, NVIDIA, Cohere, etc.	HF's internal tool, growing adoption
Available Tasks	60+ benchmarks with hundreds of subtasks	1000+ tasks (more comprehensive)
Model Backends	Broader support (GPT-NeoX, Megatron, NeMo, GGUF, OpenVINO, Neuron)	Focused on HF ecosystem, Inspect-AI
API Integration	OpenAI, Anthropic, TextSynth, Watsonx	HF Inference Endpoints, TGI, LiteLLM
Python API	Available	Available
Best For	Research reproducibility, production, specialized hardware	HF users, quick iteration, modern Python API

PICKING AN EVALUATION FRAMEWORK

A framework alone is not enough. You must know how to use it correctly. In today's chaotic LLM ecosystem, where base, chat, and reasoning models are released with little to no guidance on how benchmark results were obtained, **healthy paranoia and skepticism are essential**.

- ✔ Should I use log-likelihood evaluations for reasoning models?
- ✔ Is evaluation framework X's default configuration valid for model Y?
- ✔ How many tokens does model X need to complete its reasoning?
- ✔ Does this model require a specific prompt format?
- ✔ Should generation parameters be tuned differently?

💡 **Setting up an evaluation framework is half the battle. Learning to pilot it—and having domain-relevant benchmarks—is the other half.**

BENCHMARKS TO KNOW

Commonsense Reasoning

🏆 **ARC-Challenge (📎) →** Grade-school (primary school) **science multiple-choice QA**.

Q: Which of the following statements best explains why magnets usually stick to a refrigerator door?

A. The refrigerator door is smooth.
B. The refrigerator door contains iron.
C. The refrigerator door is a good conductor.
D. The refrigerator door has electric wires in it.

💡 **Polygl0t/ARC-poly offers translations in over 30 languages.**

BENCHMARKS TO KNOW

`Commonsense Reasoning`

🏆 **HellaSwag (📎)** → Choose the **correct next sentence** among adversarial options.

Q: Then the man writes on the snow covering the car window, and a woman in winter clothes smiles. Then …

A. “, the man adds wax to the windshield and cuts it.”
B. “, a person boards a ski lift, while two men support the head of the person wearing winter clothes.”
C. “, the man puts on a Christmas coat, knitted with netting.”
D. “, the man continues removing the snow from his car.”

💡 Polygl0t/Hellaswag-poly **offers translations in over 30 languages.**

BENCHMARKS TO KNOW

`Commonsense Reasoning`

🏆 **Global PIQA (🥏)** → Physical commonsense questions (100+ languages, 116 varieties).

Q: How do I ready a guinea pig cage for its new occupants?

A. *“Provide the guinea pig with a cage full of a few inches of bedding made of ripped paper strips, you will also need to supply it with a water bottle and a food dish.”*

B. *“Provide the guinea pig with a cage full of a few inches of bedding made of ripped jeans material, you will also need to supply it with a water bottle and a food dish.”*

BENCHMARKS TO KNOW

`World Knowledge`

🏆 **MMLU (📎)** → Knowledge on various domains (+50 disciplines). Successors: MMLU-Pro, Global-MMLU.

Q: In Aristotle’s terminology, incontinence is when:

A. *"one does not know that one’s actions are wrong."*
B. *"one knows that one’s actions are wrong, but does them anyway.“*
C. *"one knows that one’s feelings are inappropriate, and does not act on them.“*
D. *"one does the right action, but for the wrong reason.“*

💡 **Polygl0t/MMLU-poly offers translations in over 30 languages.**

BENCHMARKS TO KNOW

`Instruction Following`

🏆 **IFEval (📎)** → Follow formatting instructions. Solutions are verified using a dedicated parsing test.

>> Successors: **IFBench**.

Instruction: *Write a short blog post about a trip to Japan using fewer than 300 words.*

Instruction: *Write a letter to a friend in all lowercase, asking them to vote. Use no commas.*

💡 **Perhaps the only way to evaluate chat models. Very easy to regenerate or prevent contamination.**

BENCHMARKS TO KNOW

`Mathematics`

🏆 **GSM8K (🧶) → Grade school (primary school) math problems. Successors: MATH-500, AIME, Math-Arena.**

Q: James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?

SOLUTION: #### 624

💡 **Hard to translate. Usually requires a lot of manual curation (e.g., Polygl0t/gsm8k-pt)**

BENCHMARKS TO KNOW

Coding

🏆 **HumanEval (📎)** → Code completion. Successors: LiveCodeBench, AiderBench, SweBench.

```
def strlen(string: str) -> int:
""" Return length of given string
>>> strlen('')
0
>>> strlen('abc')
3
"""
```

```
return len(string)
```

```
def check(candidate):
    assert candidate('') == 0
    assert candidate('x') == 1
    assert candidate('asdasnakj') == 9
```

💡 **Coding evaluation sets are also good proxies for reasoning.**

BENCHMARKS TO KNOW

Long Context

🏆 **Needle in a Haystack (📎)** → Place a random fact in a long text and ask the model to retrieve it.

>> Difficult in 2023. **Solved in 2025**.

>> Successors: **RULER**, **ONERULER**.

💡 **If you have a big enough haystack (e.g., a Public domain book), you can build a RULER.**

BENCHMARKS TO KNOW

`Alignment`

🏆 **AlpacaEval (🧶)** → LM-as-a-judge test, where models are compared in a pairwise manner based on their responses to a set of instruction prompts against a reference standard (**not used that much anymore**).

>> Correlates well with human preferences (**GPT-4 Turbo as an annotator**).

>> Good for when you have a **Gold Standard**.

💡 **<u>If you want to do LLM-as-a-Judge, try to model the eval like an AlpacaEval</u>.**

~~GOOD~~ MANDATORY PRACTICES

Non-negotiables. You have to do it.

- ✓ **Take 50 random samples and manually inspect them (i.e., do it yourself). Don’t prompt an LLM to find unusual stuff in the data for you.**
  - ✔ **Are the prompts clear and unambiguous?**
  - ✔ **Are the answers correct?**
  - ✔ **Is information missing?**
  - ✔ **BE SUSPICIOUS AND PARANOID!**

~~GOOD~~ MANDATORY PRACTICES

Non-negotiables. You have to do it.

- ✓ **Perform small inspections (--limit 10 --log_samples)**
- ✓ **If your results are widely off from other reported values, investigate (go to the Issues board)**
- ✓ **Reasoning models are a pain to evaluate. If the results seem off: --limit 10 --log_samples.**
- ✓ **BE SUSPICIOUS AND PARANOID!**

RUNNING EVALS WITH THE LM EVALUATION HARNESS

- ✔ Learn how to set up the LM Evaluation Harness on an HPC cluster.
- ✔ Learn how to configure SLURM job scripts for running evaluations.
- ✔ Learn how to download and cache models from Hugging Face Hub.
- ✔ Learn how to run evaluations on specific benchmarks with few-shot settings.
- ✔ Learn how to interpret evaluation results and access output logs.

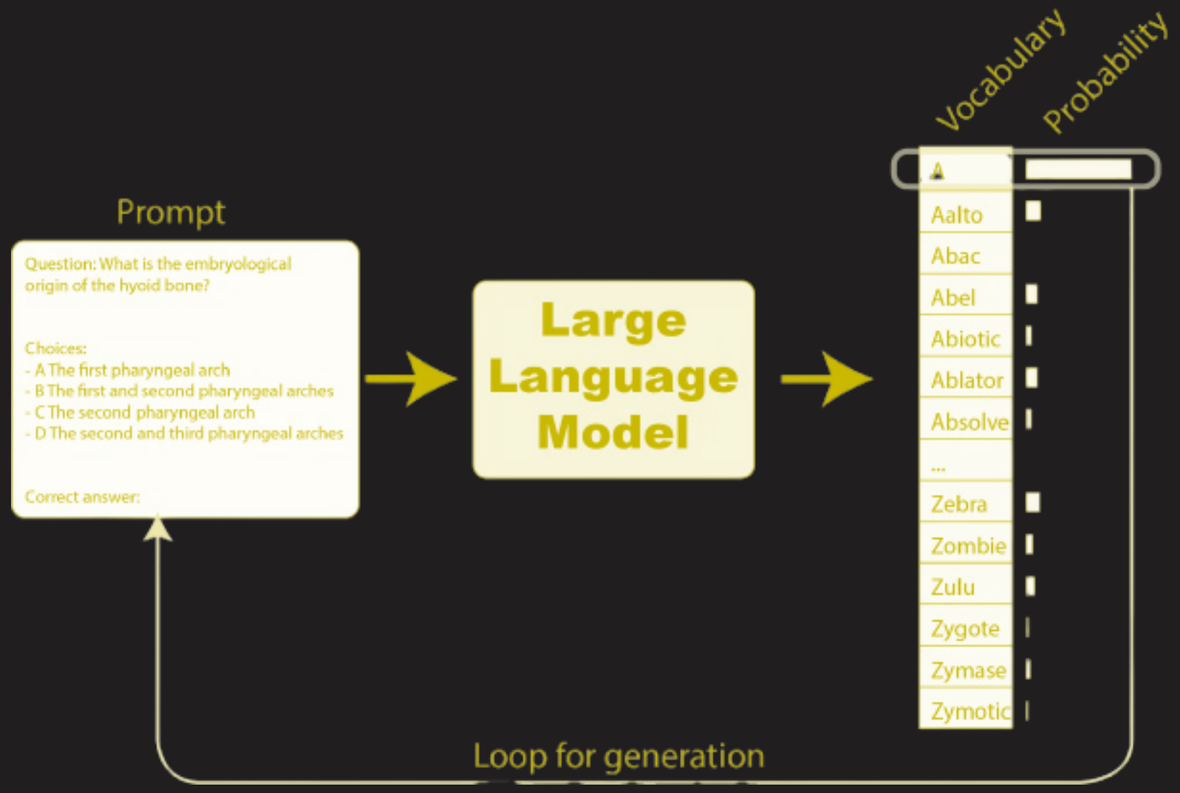
EXERCISE

A CLOSER LOOK AT EVALUATIONS

Understanding and Designing Evals

TWO MAIN EVALUATION APPROACHES

From an input prompt, an **LLM produces a probability distribution over the entire vocabulary for the next token**. To generate a continuation, we select a token from this distribution—**often the most probable one, optionally modified by controlled randomness to encourage diversity**—and append it to the prompt. This process is repeated, with each newly generated token becoming part of the context for predicting the next.



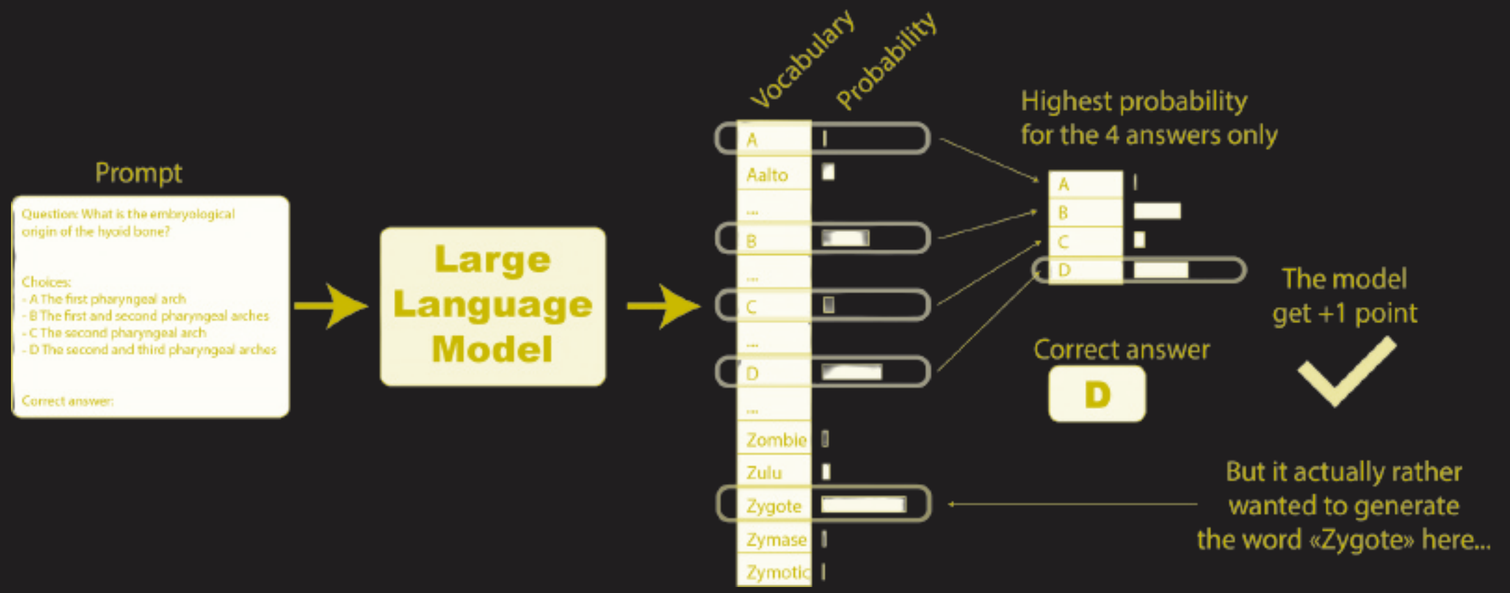
Source → The LLM Evaluation Guidebook

LOG-LIKELIHOOD EVALUATIONS

Given this autoregressive generation process, there are **two primary ways to evaluate such models**:

- ✓ **Log-likelihood evaluations:** Given a prompt and one or more candidate answers, how probable does the model consider each answer?

💡 Most models under APIs **do not return the log-probabilities**, so you'll need to use generative evaluations systematically to evaluate them.



Source → The LLM Evaluation Guidebook

LOG-LIKELIHOOD EVALUATIONS

For log-likelihood evaluation, we **estimate the probability of a specific continuation given a prompt**.

1. **Concatenate each candidate choice with the prompt and feed the resulting sequence to the LLM**, which outputs token-level logits.
2. **Keep only the logits corresponding to the choice tokens** and apply log-softmax to obtain log-probabilities (in [-∞, 0]).
3. Sum these **token log-probabilities to get the total log-likelihood** of the choice.
4. Optionally **normalize by choice length** (prevents a bias toward shorter answers).

💡 **A well-calibrated model is a model for which the correct answers have the highest probabilities.**

MULTIPLE-CHOICE AND CLOZE FORMULATIONS

Log-likelihood evaluation is commonly framed in **two ways:**

- ✓ **Multiple-Choice Format (MCF):** All answer options are explicitly listed in the prompt, and the model's likelihood of generating each option index is compared.

Example:

Prompt: *"What is the capital of France?*

A. Berlin

B. Madrid

C. Paris

Answer: → *Compare the log-likelihood of generating "A", "B", and "C".*

MULTIPLE-CHOICE AND CLOZE FORMULATIONS

- ✓ **Cloze Formulation (CF):** The prompt omits explicit choices, and we compare the likelihood of different candidate continuations.

Example:

Prompt: *“What is the capital of France?*

Candidates: “ Paris”, “ Berlin”, “ Madrid”,

Answer: → *Compare the log-likelihood of each continuation.*

MULTIPLE-CHOICE AND CLOZE FORMULATIONS

💡 Research shows that **models struggle with MCF early in training** and only acquire this skill after substantial exposure. **CF therefore provides a stronger early training signal**.

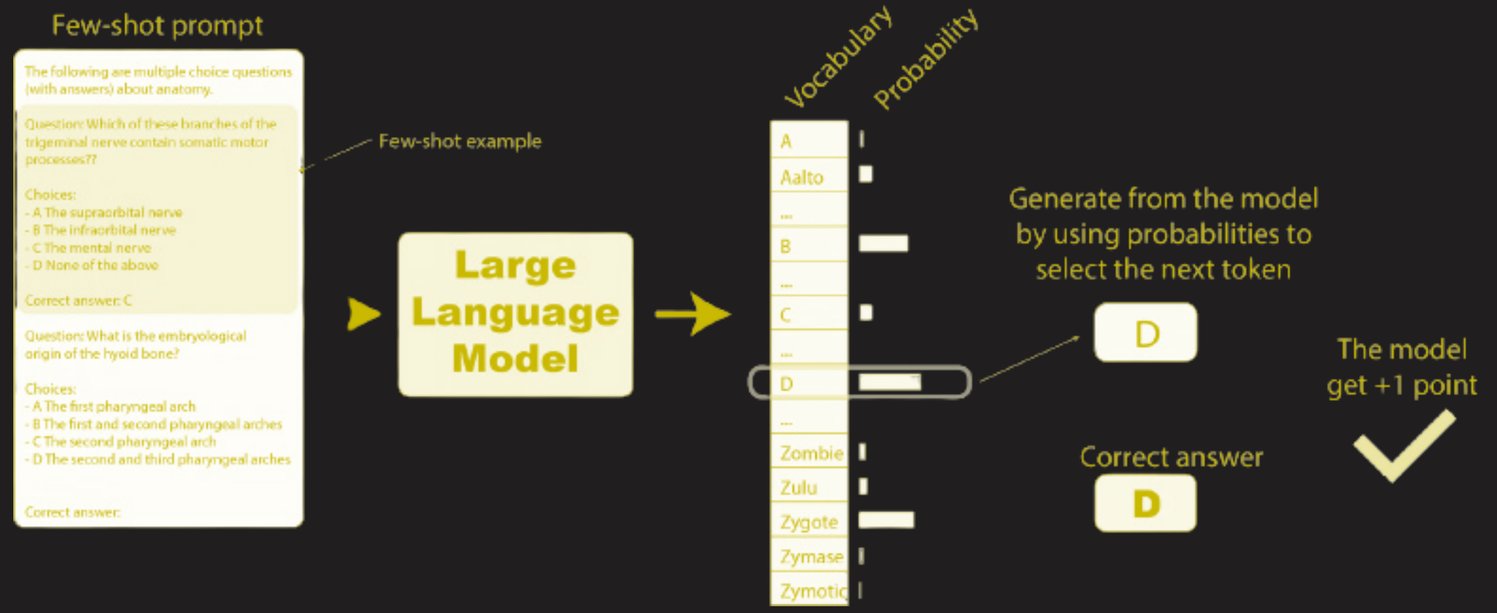
- ✔ CF for **small ablations**.
- ✔ MCF for **main runs**.

Benchmark	Tokens (B)	SNR	Spearman
Lambada	<40	18.58	0.631
HellaSwag	94	26.17	0.937
BELEBELE	660	4.78	0.949
MMLU	755	6.92	0.923

GENERATIVE EVALUATIONS

- ✓ **Generative evaluations:** Given a prompt, what text does the model actually produce?
- ✓ For **post-trained models**, this is the primary formulation.
- ✓ **Metrics:** exact match, BLEU, parsers.

💡 Freeform generation **requires substantial latent knowledge and is usually too difficult** for models during short pre-training ablations.



Source → The LLM Evaluation Guidebook

🎯 CREATING YOUR OWN EVALS

By now, you probably know **why** we run evaluations, which benchmarks exist, and when to use them (ablations, mid-training, fine-tuned models, etc.). But what happens when **nothing fits your use case**?

That's your cue to ***build your own eval.*** **Start from something battle-tested and adapt it**.

- ✔ Make the *Language-X* version of LAMBADA.
- ✔ Take an existing dataset and tweak the prompts.
- ✔ Keep the data, swap the metrics.
- ✔ Keep the task, change the framing.

💡 Reinvention is overrated. **Smart adaptation wins**. Also, don't stop at a single benchmark. **Aggregate multiple evaluations and build a small evaluation suite**.

IMPORTANT GUIDELINES

1. **Use real native speakers (aka humans)**: Evals should be designed *by* humans. Leave the LLM out of the loop—at least at the beginning.
2. **Quality > Quantity**: 100–300 carefully crafted, serious examples beat 5,000 sloppy, AI-generated ones every time.
3. **Expect iteration**: Your first version will be wrong. That's fine. Run → inspect → fix → repeat.
4. **Use proper annotation tools**: Tools like Argilla make building and managing datasets much easier.

Must Reads:

- How to set up your own annotator platform in a couple of minutes.
- A guide on annotation good practices.
- Another guide on annotation good practices.

BUILDING YOUR OWN EVALUATION BENCHMARK

- ✔ Design filtering criteria for evaluation datasets appropriate to your target language.
- ✔ Implement text processing and filtering logic in Python.
- ✔ Create evaluation datasets from raw text corpora.
- ✔ Learn how to upload datasets to Hugging Face Hub for public use.
- ✔ Learn how to create custom evaluation tasks in the LM Evaluation Harness.
- ✔ Learn how to test and validate your custom evaluation benchmark.

EXERCISE

SELECTING "GOOD" BENCHMARKS

Evaluating Evals

EVALS DURING TRAINING = SIGNALS

When you're training models, you don't just want a final score. You want to know **how training is going while it's happening**.

Different stages → different eval needs:

- ✔ Early training
- ✔ Pretraining
- ✔ Base model final eval
- ✔ Post-training

💡 At every stage, your eval must provide a **good signal**.

WHAT IS A “GOOD SIGNAL”

A task gives a **reliable signal** if its score:

- ✔ Is **better than random**
- ✔ **Improves as training progresses**
- ✔ Has **low variance across random seeds**
- ✔ Produces **stable model rankings**

💡 If your eval doesn’t behave like this… It’s probably **noise**, not signal. And if you let noise guide your development, your model will almost **certainly fail to be as performant as it could be**.

WHAT IS A “GOOD SIGNAL”

In essence, a task **must be learnable**. This is the core requirement for any eval task. The model should **learn it from data**, and that learning should be **visible over time**.

- ✔ Strong, clear signal → *we trust the benchmark!!!*
- ✔ Weak or noisy signal → *conclusions???*

💡 So we need ways to **measure the stability of that signal** across checkpoints/evaluations.

WHAT IS A “GOOD SIGNAL”

For each benchmark, try to keep a lookout for things like:

- ✔ **Max Range (Max – Min)**: Full spread of scores
  → Bigger range = *could* indicate more noise
- ✔ **Mean Absolute Change**: Avg. |score[i+1] – score[i]|
  → Higher = jumpier benchmark
- ✔ **Signal-to-Noise Ratio (SNR)**: Mean / Standard Deviation
  → Higher = real signal dominates noise
- ✔ **Spearman Correlation with Steps**: Correlation between score and training step
  → Higher/Positive = tracks learning

EVALUATING EVALUATIONS

You should complete this exercise step by step, attempting to answer each question before revealing the solutions.

- ✔ Learn to critically assess evaluations.
- ✔ Distinguish between useful and misleading evaluation signals.
- ✔ Understand when different evaluations become informative.
- ✔ Develop practical criteria for selecting training-time evaluations.

EXERCISE

BONUS CONTENT

And Other Things You Should Know

MANAGING CONTAMINATION

Assume public datasets are (or will be) contaminated.

- ✔ **Canary strings**: Insert special strings in evals so you can detect leaks (e.g., BIG-bench).
- ✔ **Encrypted or gated datasets**: Prevent easy crawling and scraping (e.g., Gate the dataset).
- ✔ **Dynamic benchmarks**: Regularly refreshed benchmarks (e.g., AIME)

💡 Contaminated datasets can **still provide a useful training signal**. Contamination ≠ useless.

SCORING FREE-FORM TEXT IS HARD

Why?

- ✔ Many valid ways to say the same thing
- ✔ Partial correctness is common
- ✔ Some tasks have **no single ground truth**

Human Evaluation

- ✔ 👍 Pros → Flexible, Strong correlation with human preferences
- ✔ 👎 Cons → Expensive, Slow

LLM-as-Judge

- ✔ 👍 Pros → Scalable, Cheap, Reproducible (with fixed settings)
- ✔ 👎 Cons → Hidden biases → Echo-chamber → Worse than expert humans for niche domains

CLOSED VS OPEN JUDGE MODELS

Closed-source

- ✔ Black box 👎
- ✔ Non-reproducible 👎
- ✔ Privacy concerns 👎
- ✔ Easy to use 👍

Open-Weights

- ✔ Reproducible 👍
- ✔ Inspectable 👍
- ✔ Local deployment 👍

Designing a Judge Prompt

- ✔ Just use MixEval

REWARD MODELS

Models trained on **human preferences** to output a score.

👍 **Pros**

- ✔ Very fast
- ✔ Deterministic
- ✔ No prompt engineering

👎 **Cons**

- ✔ Require special training
- ✔ Hard to generalize outside the domain

NEXT ON THE AGENDA

POST-TRAINING AND ALIGNMENT

Florian May & Nicholas Kluge

Lamarr Institute / Polyglot team

13:00 – 17:00

LUNCH TIME

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN